

Unit 2: Python Control Flow, Functions & Modules • Topic 1

Conditional Statements (if, if-else, if-elif-else)

Making decisions in Python programs using if, if-else, if-elif-else, and nested if statements.

So far, every program we have written executes every line in order (sequence). Real programs, however, need to make decisions — do one thing if a condition is true, and something else if it is not. This is achieved using **conditional (decision-making) statements**, which implement the **Selection** building block of logic introduced in Unit 1.

1.1 Conditions and Boolean Expressions

A **condition** is any expression that evaluates to True or False. Conditions are usually built using relational operators (`==`, `!=`, `>`, `<`, `>=`, `<=`) and combined using logical operators (`and`, `or`, `not`).

```
marks = 78
print(marks > 40)           # True
print(marks >= 40 and marks <= 100) # True
print(not (marks < 40))     # True
```

1.2 The if Statement

The simplest decision structure. The indented block runs **only if** the condition is True; otherwise it is skipped entirely.

```
if condition:
    statement(s)    # runs only when condition is True

marks = 55
if marks >= 40:
    print("You have passed")
print("Result processing complete") # always runs (outside the if-block)
```

1.3 The if-else Statement

Provides exactly **two** paths: the if block runs when the condition is True, and the else block runs when it is False. Exactly one of the two blocks always executes.

```
if condition:
    statement(s)    # runs when condition is True
else:
    statement(s)    # runs when condition is False

marks = 32
if marks >= 40:
    print("Pass")
else:
    print("Fail")
# Output: Fail
```

1.4 The if-elif-else Statement (Chained Conditions)

Used when there are **more than two** possible outcomes. Python checks each condition in order, from top to bottom, and runs the block belonging to the **first** condition that is True; all remaining conditions are skipped. If none of the conditions are True, the final else block (optional) runs.

```
if condition1:
    statement(s)
elif condition2:
    statement(s)
elif condition3:
    statement(s)
else:
    statement(s)    # runs only if ALL conditions above are False
```

Worked Example: Grading System

```
marks = 73
if marks >= 90:
    grade = "A"
elif marks >= 75:
    grade = "B"
elif marks >= 60:
    grade = "C"
elif marks >= 40:
    grade = "D"
else:
    grade = "F"
print("Grade:", grade)    # Grade: C
```

COMMON PITFALL: Order matters in if-elif-else. If the conditions above were written from smallest to largest (marks >= 40 first), every mark of 40 or above would incorrectly get grade 'D' because Python stops at the first True condition.

1.5 Nested if Statements

An if (or if-else) statement placed **inside** another if/else block is called a **nested if**. It is used when a decision itself depends on another decision.

```
age = 20
has_id = True
if age >= 18:
    if has_id:
        print("Entry allowed")
    else:
        print("ID required for entry")
else:
    print("Entry not allowed - underage")
```

Nested ifs work, but deeply nested code (many levels) becomes hard to read. Where possible, combine conditions using and/or instead, e.g. if age >= 18 and has_id:.

1.6 if vs if-else vs if-elif-else — Quick Comparison

Structure	Number of outcomes	When to use
if	1 (runs or is skipped)	Only one condition needs handling

Structure	Number of outcomes	When to use
if-else	Exactly 2	Exactly two mutually exclusive outcomes
if-elif-else	3 or more	Several mutually exclusive outcomes / ranges
Nested if	Depends on structure	A decision depends on another decision

Practice Questions

A. Multiple Choice Questions

- Which statement runs one of exactly two blocks depending on a condition?
(A) if (B) if-else (C) for (D) while
- In an if-elif-else chain, how many blocks execute at most?
(A) All of them (B) Exactly one (C) Exactly two (D) None ever execute
- What will the following print? `x = 15 \n if x > 20: print('A') \n elif x > 10: print('B') \n else: print('C')`
(A) A (B) B (C) C (D) Nothing
- An if statement placed inside another if statement is called:
(A) Chained if (B) Nested if (C) Compound if (D) Recursive if
- Which of these is a valid boolean condition?
(A) `x = 5` (B) `x == 5` (C) `x := 5` (D) `let x be 5`

B. True / False

- In an if-elif-else chain, Python evaluates conditions from top to bottom and stops at the first True one.
- An if-else statement can have more than one else block.
- Nested if statements are used when one decision depends on the outcome of another.
- The else block in if-elif-else is compulsory.

C. Short Answer Questions

- Differentiate between if-else and if-elif-else with a suitable example of each.
- Why does the order of conditions matter in an if-elif-else chain? Illustrate with an example.
- What is a nested if statement? When would you prefer it over combining conditions with 'and'?
- Write the general syntax of an if-elif-else statement.

D. Long Answer / Descriptive Questions

- Write a Python program using if-elif-else that categorises a person's BMI (weight/height²) into Underweight (<18.5), Normal (18.5–24.9), Overweight (25–29.9), and Obese (≥30), and explain why the conditions must be checked in this specific order.
- Explain, with an example program, how nested if statements can be used to check whether a given year is a leap year (divisible by 4, but century years must be divisible by 400).

E. Programming Problems

- Write a Python program that takes three numbers as input and prints the largest among them using nested if statements.
- Write a Python program that takes a character as input and, using if-elif-else, prints whether it is a vowel or a consonant (assume the input is always a single alphabet letter).
- Write a Python program that takes the cost price and selling price of an item and prints whether the transaction resulted in profit, loss, or no profit/no loss, along with the amount.
- Write a Python program that takes a number and checks, using if-elif-else, whether it is positive and even, positive and odd, negative and even, negative and odd, or zero.

Answer Key (A & B): MCQ: 1-B, 2-B, 3-B, 4-B, 5-B | T/F: 6-True, 7-False, 8-True, 9-False